

UNIT 2

Introduction to Web Servers and Node.js Development

Topics Covered in Unit 2

1. Introduction to Web Servers
2. JavaScript on the Desktop with Node.js
3. NPM – Node Package Manager
4. Serving Files with the HTTP Module
5. Introduction to the Express Framework
6. Server-Side Rendering with Templating Engines
7. Static Files in Express
8. async/await in Node.js
9. Fetching JSON Data from Express

1. Introduction to Web Servers

1.1 The World Wide Web — A Brief Background

Before we dive into web servers, it is important to understand what the World Wide Web (WWW) is and how it came to exist. The World Wide Web was invented by Sir Tim Berners-Lee in 1989 while working at CERN (a physics research organization in Switzerland). He created it as a way for scientists to share information with each other over the internet.

Today, the World Wide Web is a massive collection of billions of web pages and web applications that are connected to each other through hyperlinks. When you open a browser and visit any website — whether it is Google, YouTube, Facebook, or your bank's website — you are using the World Wide Web.

The Web works on a simple idea: there are computers called servers that store and share information, and there are computers called clients (like your laptop or phone) that request and display that information.

1.2 What is a Web Server?

Definition: Web Server

A web server is a software program that runs continuously on a computer (also called a server machine) and is responsible for accepting incoming requests from clients over the internet or a network, processing those requests, and sending back appropriate responses. The response may be an HTML web page, an image, a video, a JSON data file, or any other type of resource. Examples of popular web server software include Apache HTTP Server, Nginx, Microsoft IIS, and Node.js with the Express framework.

In simple terms, a web server is like a librarian. When you go to a library and ask for a specific book (request), the librarian finds that book and hands it to you (response). Similarly, when your browser asks for a web page, the web server finds and sends that page back to the browser.

Real-World Analogy:

Think of a web server as a restaurant. You (the client/browser) sit at a table and give your order to the waiter (HTTP Request). The waiter carries your order to the kitchen (server). The kitchen prepares the food (processes the request). The waiter brings the food back to your table (HTTP Response). You eat the food (the browser displays the page). In this analogy, the restaurant is the web server, the waiter is HTTP protocol, the kitchen is the backend code, and the food is the response data (HTML/JSON).

1.3 The Client-Server Architecture

Definition: Client-Server Architecture

Client-Server Architecture is a computing model in which tasks or workloads are distributed between two types of participants: the client and the server. The client is the program that initiates communication and requests services or resources. The server is the program that listens for incoming requests, processes them, and returns responses. This model forms the foundation of how the internet and web applications work.

In web development, the client is almost always a web browser (like Google Chrome, Mozilla Firefox, or Microsoft Edge). The browser runs on the user's device and is responsible for:

- Sending requests to the server for web pages, images, data, etc.
- Receiving the response from the server
- Displaying the content (HTML, CSS, JavaScript) to the user

The server, on the other hand, runs on a powerful computer (often in a data center) and is responsible for:

- Listening for incoming requests 24 hours a day, 7 days a week
- Processing the request — this might involve reading from a database, performing calculations, or simply reading a file
- Sending the appropriate response back to the client

1.4 What is HTTP?

Definition: HTTP (HyperText Transfer Protocol)

HTTP stands for HyperText Transfer Protocol. It is a set of rules and conventions that govern how data is transmitted between a web client (browser) and a web server over the internet. HTTP is the foundation of all data communication on the World Wide Web. It is a request-response protocol, meaning the client always initiates a request and the server always sends a response. HTTP operates on top of the TCP/IP (Transmission Control Protocol/Internet Protocol) network protocol.

Whenever you type a URL (Uniform Resource Locator) in your browser and press Enter, the following process happens:

10. Your browser creates an HTTP Request message containing the URL, the type of request (GET), and other header information.
11. This request is sent over the internet to the web server associated with that URL.
12. The web server receives the request and processes it.
13. The web server creates an HTTP Response message containing the requested content (HTML page, JSON data, etc.) along with a status code.

14. The HTTP Response is sent back over the internet to your browser.
15. Your browser reads the response and displays the web page on your screen.

1.4.1 HTTP Request Methods

An HTTP Request Method (also called HTTP Verb) tells the server what kind of operation the client wants to perform. There are several HTTP methods, but the four most commonly used in web development are:

HTTP Method	Full Name	Purpose	Real-World Example
GET	Retrieve	Fetch or read data from the server. Does not modify any data.	Opening a web page, loading a list of products on an e-commerce site
POST	Create	Send new data to the server to create a new resource.	Submitting a registration form, creating a new blog post
PUT	Update/Replace	Send data to update an existing resource on the server.	Editing and saving your profile information
DELETE	Delete	Request the server to delete a specific resource.	Deleting a social media post, removing an account

Note for Students:

In the context of this unit, you will mostly work with GET and POST requests. GET is used to retrieve data (like loading a webpage or getting a list of students from a database). POST is used to submit or send data to the server (like submitting a login form or creating a new record).

1.4.2 HTTP Status Codes

Definition: HTTP Status Code

An HTTP Status Code is a 3-digit number that a web server sends back in every HTTP response to indicate the result of the request. The first digit of the status code indicates the category of the response. Status codes help both the client (browser) and developers understand whether a request was successful, or if something went wrong.

Status codes are grouped into five categories based on their first digit:

- **1xx (Informational):** The request was received and is being processed.
- **2xx (Success):** The request was received, understood, and completed successfully.
- **3xx (Redirection):** Further action needs to be taken to complete the request. Usually means the page has moved.

- **4xx (Client Error):** There was an error caused by the client (for example, requesting a page that does not exist).
- **5xx (Server Error):** The server failed to fulfil a valid request due to an error on the server side.

Status Code	Meaning	When It Occurs
200 OK	Request was successful.	Page loaded correctly, data fetched successfully
201 Created	A new resource was successfully created.	After a successful POST request (new user registered)
301 Moved Permanently	The resource has been permanently moved.	Website changed its URL permanently
400 Bad Request	The server cannot understand the request due to wrong syntax.	Client sent malformed data
401 Unauthorized	Authentication is required but not provided.	Trying to access a private page without logging in
403 Forbidden	Server understands but refuses to allow access.	Trying to access admin panel without permission
404 Not Found	The requested resource does not exist on the server.	Typing a wrong URL or page deleted
500 Internal Server Error	Server encountered an unexpected condition.	Bug or crash in the server-side code

Important Point:

As a developer, you will use these status codes to communicate the result of operations to the client. For example, when a user successfully creates an account, you send back status code 201. When a requested record is not found, you send back 404. When there is a server-side bug, a 500 status code is sent automatically.

1.5 URL Structure — Understanding Web Addresses

Definition: URL (Uniform Resource Locator)

A URL, or Uniform Resource Locator, is a complete web address used to locate a specific resource on the internet. Every resource on the web — whether it is a web page, an image, a video, or an API endpoint — has a unique URL. A URL provides all the information needed to access a resource: the protocol to use, the server to contact, the path to the resource, and any additional query parameters.

A URL is composed of multiple parts. Consider this example URL:

Example URL:

```
https://www.example.com:3000/students/profile?id=42&city=surat
```

URL Part	Value in Example	What It Means
Protocol	https://	The communication protocol to use (HTTP or HTTPS)
Domain Name	www.example.com	The human-readable address of the server
Port Number	:3000	The specific port on the server (optional; default is 80 for HTTP, 443 for HTTPS)
Path	/students/profile	The specific resource/page being requested on the server
Query String	?id=42&city=surat	Additional parameters sent to the server. Each param is key=value, separated by &

1.6 HTTPS — The Secure Version of HTTP

Definition: HTTPS (HyperText Transfer Protocol Secure)

HTTPS is a secure version of HTTP that encrypts the data exchanged between the client and the server using SSL/TLS (Secure Sockets Layer / Transport Layer Security) encryption. This ensures that even if someone intercepts the data while it is traveling across the internet, they cannot read it. HTTPS is mandatory for websites that handle sensitive information such as passwords, credit card numbers, and personal data.

Example:

When you log in to your bank's website or enter your credit card details on an e-commerce site, notice that the URL starts with 'https://' and there is a padlock icon in the browser address bar. This confirms that the data you are sending is encrypted and secure. In contrast, if a website uses 'http://', the data is transmitted as plain text and can be intercepted.

2. JavaScript on the Desktop with Node.js

2.1 A Brief History — Why Node.js Was Created

For a long time in the history of web development, JavaScript was considered only a frontend (client-side) language. This means JavaScript could only run inside a web browser. If you wanted to build the server side of a web application — that is, the part that handles requests, connects to databases, and sends responses — you had to use a completely different language such as PHP, Java, Python, or Ruby.

This meant a web developer had to know at least two different programming languages: one for the frontend (JavaScript) and one for the backend (PHP, Python, etc.). In 2009, a software engineer named Ryan Dahl created Node.js to solve this problem. His idea was simple but revolutionary: make it possible to run JavaScript outside the browser, directly on the server. This meant developers could use JavaScript for both the frontend and the backend, reducing the learning curve significantly.

2.2 What is Node.js?

Definition: Node.js

Node.js is an open-source, cross-platform, server-side JavaScript runtime environment built on Google Chrome's V8 JavaScript engine. It allows developers to execute JavaScript code outside of a web browser, directly on a computer or server. Node.js uses an event-driven, non-blocking I/O (Input/Output) model that makes it efficient and suitable for building scalable network applications such as web servers, APIs, real-time chat applications, and streaming services. Node.js was created by Ryan Dahl and was first released in 2009.

Let us break down this definition into simpler terms:

- **Open-source:** The code of Node.js itself is freely available for anyone to see, use, and contribute to.
- **Cross-platform:** Node.js can run on Windows, macOS, and Linux operating systems.
- **Runtime Environment:** A runtime environment provides all the necessary tools and libraries to execute a programming language. Node.js provides the environment to execute JavaScript code on the server.
- **V8 JavaScript Engine:** V8 is the same engine that powers Google Chrome's browser. It is extremely fast because it compiles JavaScript directly into machine code before executing it.
- **Non-blocking I/O:** This means Node.js does not wait for one operation to complete before starting the next. It handles multiple operations simultaneously, making it very fast and efficient.

2.3 JavaScript in Browser vs JavaScript with Node.js

To truly understand Node.js, it is essential to understand the difference between JavaScript running in a browser and JavaScript running in Node.js:

Feature	JavaScript in Browser	JavaScript with Node.js
Where it executes	Inside the web browser (Chrome, Firefox, Safari)	Directly on the computer or server machine
Can access local files?	No — browsers prevent this for security reasons	Yes — can read, write, and delete files on the disk
Can create a web server?	No	Yes — can listen for HTTP requests and respond
Can connect to databases?	No	Yes — can connect to MySQL, MongoDB, etc.
Has access to OS features?	No — limited sandbox environment	Yes — can access network, file system, processes
Primary use	Building interactive web page elements (buttons, forms, animations)	Building the backend (server logic, APIs, databases)
Available APIs	DOM (document), window, fetch, localStorage, etc.	fs (file system), http, path, os, crypto, etc.

Key Takeaway:

Node.js does NOT replace JavaScript in the browser. Both still exist and serve different purposes. The browser runs JavaScript for frontend interactions, while Node.js runs JavaScript for backend/server-side operations. Together, they form the foundation of a Full Stack JavaScript application.

2.4 Installing Node.js

Installing Node.js is a simple process. Follow these steps:

16. Open a web browser and go to the official Node.js website: <https://nodejs.org>
17. You will see two options: LTS (Long Term Support) and Current. Always download the LTS version — it is the stable, production-ready version recommended for most users.
18. Download the installer appropriate for your operating system (Windows, macOS, or Linux).
19. Run the installer and follow the on-screen instructions. Keep all default settings.
20. Once installation is complete, verify it by opening a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and typing the following commands:

Verify installation by checking version numbers:

```
node --version  
npm --version
```

If you see version numbers printed (for example, v20.11.0 and 10.2.3), Node.js and NPM have been successfully installed on your computer.

What is LTS?

LTS stands for Long Term Support. LTS versions of Node.js receive bug fixes and security updates for a longer period (usually 30 months). This makes them more stable and reliable for production use. The 'Current' version has the latest features but may have bugs and receives shorter support.

2.5 How Node.js Works — The Event Loop

Definition: Event Loop

The Event Loop is the mechanism at the heart of Node.js that enables non-blocking, asynchronous programming. Rather than waiting for slow operations (like reading a file or querying a database) to complete before moving on, Node.js registers a callback function for each such operation and continues executing other code. When the slow operation finishes, Node.js places the callback in an event queue. The event loop continuously checks this queue and executes the callbacks in order.

To understand why this matters, consider this scenario. Imagine a restaurant with only one waiter. There are two ways this waiter can work:

- **Blocking (Synchronous) Approach:** The waiter takes Order 1 to the kitchen, then stands at the kitchen window and waits until the food is ready (which takes 10 minutes), then delivers it, then takes Order 2, waits again, and so on. Other customers must wait a very long time.
- **Non-Blocking (Asynchronous / Node.js) Approach:** The waiter takes Order 1 to the kitchen, does NOT wait, comes back and takes Order 2, then Order 3. When Order 1 is ready, the kitchen notifies the waiter (event), and the waiter delivers it. This way, all customers get served much faster.

Node.js works like the second waiter. It never stands idle waiting for a slow operation. Instead, it registers a callback (a function to call when done), moves on to the next task, and comes back to handle the result when it is ready. This is why Node.js can handle thousands of simultaneous connections with very high efficiency.

2.6 Writing and Running Your First Node.js Program

Let us write a simple Node.js program to get started. Follow these steps carefully:

21. Create a new folder on your computer, for example: my-node-projects
22. Inside that folder, create a new file called hello.js
23. Open the file in a text editor (VS Code is recommended) and write the following code:

File: hello.js

```
// This is a comment. Comments are ignored by Node.js
// The console.log() function prints a message to the terminal

console.log('Hello! This is my first Node.js program.');
```



```
// Variables work the same way as in browser JavaScript
let studentName = 'Priya';
let semester = 2;

console.log('Student Name: ' + studentName);
console.log('Semester: ' + semester);
```



```
// Arithmetic also works the same way
let sum = 10 + 20;
console.log('10 + 20 = ' + sum);
```

24. Open a terminal (Command Prompt or Terminal), navigate to the folder where hello.js is saved using the 'cd' command, and run the file using Node.js:

Running the program in terminal:

```
cd my-node-projects
node hello.js
```

Expected Output:

```
Hello! This is my first Node.js program.
Student Name: Priya
Semester: 2
10 + 20 = 30
```

Explanation:

The 'node' command tells the operating system to execute the JavaScript file using the Node.js runtime. The 'hello.js' is the name of the file to execute. Node.js reads the file, interprets the JavaScript code line by line, and executes it. The console.log() function outputs messages to the terminal screen, just like it outputs to the browser console in frontend JavaScript.

2.7 Node.js Built-in Modules

Definition: Module

In Node.js, a module is a self-contained file or package that encapsulates related code, functions, and data. Modules allow developers to organise code into separate files and reuse functionality across different parts of an application. Node.js has three types of modules: (1) Built-in Core Modules that come pre-installed with Node.js, (2) Third-Party Modules that are installed via NPM, and (3) Custom Modules that are created by the developer.

Node.js comes with many built-in modules that you can use without installing anything. To use a built-in module, you use the `require()` function:

Using the built-in 'os' module (gives information about the operating system):

```
const os = require('os');    // Load the 'os' module

console.log('Operating System Platform: ' + os.platform());
console.log('CPU Architecture: ' + os.arch());
console.log('Total RAM: ' + Math.round(os.totalmem() / 1024 / 1024) + ' MB');
console.log('Computer Hostname: ' + os.hostname());
```

Built-in Module	Purpose	Example Usage
http	Create web servers and handle HTTP requests/responses	<code>require('http')</code>
fs	Read, write, delete, and manage files on the computer	<code>require('fs')</code>
path	Work with file and directory paths across different OS	<code>require('path')</code>
os	Get information about the operating system and hardware	<code>require('os')</code>
events	Create and handle custom events	<code>require('events')</code>
url	Parse and format URL strings	<code>require('url')</code>
crypto	Perform cryptographic operations (hashing, encryption)	<code>require('crypto')</code>

3. NPM – Node Package Manager

3.1 What is NPM?

Definition: NPM (Node Package Manager)

NPM stands for Node Package Manager. It is a command-line tool and an online registry that comes bundled with Node.js. NPM serves two primary purposes: First, it is a command-line utility that allows developers to install, update, and manage external libraries (called packages) for Node.js projects. Second, it is the world's largest online repository of open-source JavaScript packages, hosted at <https://www.npmjs.com>, which contains over 2 million packages created by developers worldwide.

Think of NPM like a massive App Store, but for code libraries. Instead of building every functionality from scratch, a developer can search for an existing package that already does the job, install it with a single command, and use it immediately in their project.

Real-World Analogy:

Imagine you are building a house. You could make every brick, door, window, and tile yourself from raw materials — but that would take years. Or you could buy pre-made, high-quality doors, windows, and tiles from a store and just assemble them. NPM is that store for Node.js developers. Need to validate emails? There is a package. Need to hash passwords? There is a package. Need to connect to MongoDB? There is a package.

3.2 What is a Package?

Definition: Package

In the context of NPM and Node.js, a package is a reusable piece of code published to the NPM registry. A package typically contains one or more JavaScript files that provide specific functionality, along with a `package.json` file that describes the package's name, version, dependencies, and other metadata. When you install a package using NPM, it is downloaded from the NPM registry and stored in your project's `node_modules` folder.

3.3 The package.json File

Definition: package.json

package.json is a JSON (JavaScript Object Notation) configuration file that is placed at the root of every Node.js project. It serves as the identity and configuration document of the project. It stores essential information such as the project name, version, description, author, and most importantly, the list of all packages (dependencies) that the project requires. The package.json file allows any developer to reproduce the exact same project environment by simply running 'npm install', which reads the file and downloads all listed packages.

To create a package.json file, navigate to your project folder in the terminal and run:

Creating a new package.json file:

```
npm init -y
```

The -y flag automatically answers 'yes' to all questions and creates the file with sensible default values. Here is what a typical package.json looks like:

Sample package.json file:

```
{
  "name": "my-web-app",
  "version": "1.0.0",
  "description": "My first Node.js web application",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "author": "Your Name",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2",
    "ejs": "^3.1.9"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Field	Purpose
name	The name of your project. Should be lowercase with no spaces.
version	The current version of your project, following semantic versioning (major.minor.patch)
description	A short, human-readable description of what the project does

main	The entry point file — the file Node.js runs when the project starts
scripts	Custom command shortcuts. 'npm start' runs the 'start' script, 'npm run dev' runs the 'dev' script
dependencies	Packages required for the application to run in production
devDependencies	Packages required only during development, not needed in production (e.g., testing tools, nodemon)

3.4 Installing and Managing Packages

Installing a package with NPM is done using the 'npm install' command (can be shortened to 'npm i'):

Common NPM commands:

```
// Install a package and add it to dependencies in package.json
npm install express

// Install a package only for development (not needed in production)
npm install nodemon --save-dev

// Install all packages listed in package.json (used when cloning a project)
npm install

// Uninstall/remove a package
npm uninstall express

// Check the installed version of a package
npm list express

// Update a package to its latest version
npm update express
```

3.5 The node_modules Folder

Definition: node_modules

The node_modules folder is a directory automatically created by NPM when you install packages. It contains the actual code (JavaScript files) of all the installed packages and their own dependencies. This folder can become very large (sometimes containing hundreds of MB of files) because a single package may depend on many other packages, which in turn depend on more packages. This is why the node_modules folder should NEVER be shared, uploaded to GitHub, or committed to version control.

Important Point:

CRITICAL RULE: Never upload the `node_modules` folder to GitHub or any version control system. Instead, add `'node_modules/'` to a file called `'.gitignore'` in your project root. When another developer clones your project, they simply run `'npm install'`, and NPM will automatically download and recreate the entire `node_modules` folder on their computer using the list of packages in `package.json`.

3.6 Semantic Versioning

When you look at a package version number like 4.18.2, this follows a standard called Semantic Versioning (SemVer). Each number has a specific meaning:

- **4 (Major Version):** Increased when there are incompatible, breaking changes that may require you to rewrite some of your code.
- **18 (Minor Version):** Increased when new features are added in a backward-compatible way (your existing code still works).
- **2 (Patch Version):** Increased for backward-compatible bug fixes and security patches.

In `package.json`, you may see version numbers with special characters like `^` or `~`:

- **`^4.18.2` (Caret):** Install version 4.18.2 or any newer MINOR/PATCH version, but do not upgrade to version 5.x.x (major).
- **`~4.18.2` (Tilde):** Install version 4.18.2 or any newer PATCH version, but do not upgrade to 4.19.x (minor) or higher.

4. Serving Files with the HTTP Module

4.1 What is the HTTP Module?

Definition: http Module

The http module is a built-in Node.js core module that provides functionality to create HTTP web servers and make HTTP client requests. Since it is a built-in module, no installation is needed — it comes pre-packaged with Node.js. The http module provides the lowest level of HTTP functionality in Node.js, giving developers full control over how requests are handled and responses are constructed. It is the foundation upon which all Node.js web frameworks (including Express) are built.

To use the http module in your Node.js code, you simply require it at the top of your file:

Importing the http module:

```
const http = require('http');
```

4.2 Creating a Basic Web Server

Let us now create a web server using the http module step by step. This will help you understand exactly what happens under the hood every time a web server handles a request:

File: server.js — A basic web server using the http module

```
// Step 1: Import the built-in http module
const http = require('http');

// Step 2: Create the server
// The createServer() function takes a callback function as its argument.
// This callback is called every time a new HTTP request arrives at the
// server.
// The callback receives two objects:
//   - request (req): Contains all information about the incoming request
//   - response (res): Used to send the response back to the client
const server = http.createServer(function(request, response) {

    // Step 3: Set the response status code and headers
    // writeHead(statusCode, headers) sets the HTTP status code and response
    // headers
    // 200 means 'OK' (success)
    // 'Content-Type: text/html' tells the browser this is an HTML document
    response.writeHead(200, { 'Content-Type': 'text/html' });

    // Step 4: Write the response body
```

```
// The write() method adds content to the response body
response.write('<html>');
response.write('<head><title>My First Server</title></head>');
response.write('<body>');
response.write('<h1>Hello! Welcome to my Node.js Web Server!</h1>');
response.write('<p>This page was created by Node.js on the server
side.</p>');
response.write('</body></html>');

// Step 5: End the response – ALWAYS call end() to signal response is
complete
// Without end(), the browser will keep waiting forever
response.end();
});

// Step 6: Start the server and tell it to listen on port 3000
// The callback function runs once when the server successfully starts
server.listen(3000, function() {
  console.log('Server started! Visit http://localhost:3000 in your
browser.');
```

To run this server, open a terminal, navigate to the folder containing server.js, and run:

Running the server:

```
node server.js
```

Now open a web browser and type <http://localhost:3000> in the address bar. You will see your web page.

4.3 Understanding localhost and Port Numbers

Definition: localhost

localhost is a special hostname that refers to the current computer itself. It is always mapped to the IP address 127.0.0.1. When a developer runs a web server on their own computer and wants to test it, they access it using the URL <http://localhost> (instead of a real domain name). This is because the server is running locally — on the developer's own machine — and has not yet been deployed to the internet.

Definition: Port Number

A port number is a 16-bit integer (ranging from 0 to 65535) that identifies a specific process or service running on a computer. Think of your computer as a large building with thousands of numbered doors (ports). Each door leads to a different service or program. When you run a web server on port 3000, it means the server is 'listening' at door number 3000 for incoming

connections. Port 80 is the default port for HTTP, port 443 is the default for HTTPS, and port 3000 is commonly used during development.

Port Number	Standard Use
80	Default HTTP port — when you visit <code>http://www.example.com</code> , the browser automatically uses port 80
443	Default HTTPS port — used for all secure (encrypted) web traffic
3000	Commonly used during Node.js/Express development as a non-privileged port
8080	Alternative HTTP port often used for development or testing
27017	Default port for MongoDB database
3306	Default port for MySQL database

4.4 Handling Different URL Routes

In a real web application, users can visit different pages. The `req.url` property gives us the URL path that was requested, allowing us to serve different content for different pages:

server.js — Serving different pages based on URL:

```
const http = require('http');
const fs = require('fs');    // Built-in File System module

const server = http.createServer(function(req, res) {

  console.log('Request received for URL: ' + req.url);

  if (req.url === '/' || req.url === '/home') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Welcome to the Home Page!</h1>');
  } else if (req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>About Us</h1><p>We are learning Node.js!</p>');
  } else if (req.url === '/contact') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Contact Us</h1><p>Email: info@example.com</p>');
  } else {
    // If no route matches, send a 404 Not Found response
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.end('<h1>404 - Page Not Found!</h1>');
  }
});
```

```
});  
  
server.listen(3000, () => {  
  console.log('Multi-page server running on http://localhost:3000');  
});
```

Observation:

You can see that building a multi-page application using only the raw http module requires many if-else conditions. This approach quickly becomes messy and difficult to manage as the application grows. This is the exact problem that the Express framework (Topic 5) solves by providing a clean, organized routing system.

4.5 Reading and Serving HTML Files

Instead of writing HTML directly inside JavaScript strings, the best practice is to store HTML in separate .html files and read them using Node.js's built-in fs (File System) module:

Reading and serving an HTML file:

```
const http = require('http');  
const fs = require('fs');  
  
const server = http.createServer(function(req, res) {  
  // Read the HTML file from the disk asynchronously  
  fs.readFile('./index.html', function(error, fileData) {  
    if (error) {  
      // If the file could not be read, send a 500 error  
      res.writeHead(500, { 'Content-Type': 'text/plain' });  
      res.end('Internal Server Error: Could not read file.');    } else {  
      // If the file was read successfully, send it as the response  
      res.writeHead(200, { 'Content-Type': 'text/html' });  
      res.end(fileData); // fileData contains the HTML file content  
    }  
  });  
});  
  
server.listen(3000, () => console.log('Server running!'));
```

5. Introduction to the Express Framework

5.1 What is a Web Framework?

Definition: Web Framework

A web framework is a pre-built collection of tools, libraries, and conventions that simplify and accelerate the process of building web applications and APIs. A web framework provides a structured foundation and reusable components for common web development tasks such as routing, handling HTTP requests and responses, managing sessions, rendering views, and interfacing with databases. Using a framework saves developers from writing repetitive boilerplate code from scratch.

5.2 What is Express.js?

Definition: Express.js (Express)

Express.js, commonly referred to simply as Express, is a minimal, fast, and unopinionated web application framework for Node.js. It is built on top of Node.js's built-in http module and provides a thin layer of fundamental web application features, including robust routing, middleware support, HTTP utility methods, and template engine integration, without obscuring Node.js features. Express is the most widely used Node.js web framework and is a core component of the popular MEAN and MERN full-stack development stacks.

Express was created by TJ Holowaychuk and first released in 2010. According to the official npm registry, Express is downloaded tens of millions of times every week, making it one of the most popular open-source packages in the world.

The term 'unopinionated' means Express does not force you to follow a specific directory structure, design pattern, or coding style. You have complete freedom to organise your application however you like. This is different from 'opinionated' frameworks like Ruby on Rails or Django, which enforce strict conventions.

5.3 Installing Express

Since Express is a third-party package (not a built-in Node.js module), it must be installed using NPM. Make sure you have already initialized your project with `npm init -y` before installing Express:

Installing Express:

```
npm install express
```

After running this command, you will notice that:

- A `node_modules/` folder is created containing the Express package and all its dependencies
- The `package.json` file is updated to include `express` under the dependencies section

5.4 Your First Express Application

File: `app.js` — Hello World with Express

```
// Step 1: Import the express package
const express = require('express');

// Step 2: Create an Express application instance
// app is the central object through which all Express features are accessed
const app = express();

// Step 3: Define a route
// app.get(path, handlerFunction) means:
//   'When a GET request is received for this path, run this function'
// req = request object (contains info about the incoming request)
// res = response object (used to send back the response)
app.get('/', function(req, res) {
  res.send('Hello! Welcome to my first Express application!');
});

// Step 4: Start the server and listen for incoming connections on port 3000
app.listen(3000, function() {
  console.log('Express app running at http://localhost:3000');
  console.log('Press Ctrl+C to stop the server.');
```

What is happening here?

When you run `'node app.js'` and visit `http://localhost:3000` in your browser, your browser sends a GET request to the server. Express receives this request, matches it to the route defined by `app.get('/')`, calls the handler function, which calls `res.send()` to send the text back as the HTTP response. Your browser receives this text and displays it on screen.

5.5 Routing in Express

Definition: Routing

Routing refers to the mechanism by which a web application determines how it should respond to a specific client request based on the URL (path) and HTTP method of that request. In Express, each route consists of a path (the URL pattern to match), an HTTP method (GET, POST, PUT, DELETE), and one or more handler functions that are executed when the route is matched.

Defining multiple routes in Express:

```
const express = require('express');
const app = express();

// Route for the home page
app.get('/', function(req, res) {
  res.send('<h1>Home Page</h1><a href="/about">Go to About</a>');
});

// Route for the about page
app.get('/about', function(req, res) {
  res.send('<h1>About Us</h1><p>We are learning Full Stack Development!</p>');
});

// Route for the contact page
app.get('/contact', function(req, res) {
  res.send('<h1>Contact</h1><p>Email us at: info@example.com</p>');
});

// Handling 404 - This MUST be the LAST route defined
app.use(function(req, res) {
  res.status(404).send('<h1>404 Error - Page Not Found</h1>');
});

app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

5.5.1 Route Parameters

Definition: Route Parameter

A route parameter is a named segment of a URL that acts as a placeholder for a dynamic value. In Express, route parameters are defined by prefixing a colon (:) before the parameter name in the route path. When a request is received, Express extracts the actual value from the URL and makes it available through the `req.params` object. Route parameters are used to create dynamic routes that can handle different resources based on an identifier.

Using route parameters:

```
// :id is a route parameter - it can be any value
app.get('/student/:id', function(req, res) {
  const studentId = req.params.id; // Extract the value from URL
  res.send('You requested information for Student ID: ' + studentId);
});

// Multiple parameters in one route
app.get('/students/:year/:division', function(req, res) {
```

```

const year = req.params.year;
const division = req.params.division;
res.send('Year: ' + year + ', Division: ' + division);
});

// Example: visiting /student/42 will show: You requested information for
Student ID: 42
// Example: visiting /students/2/A will show: Year: 2, Division: A

```

5.5.2 Query Parameters

Definition: Query Parameters

Query parameters (also called query strings) are key-value pairs appended to the end of a URL, starting with a question mark (?). Multiple query parameters are separated by an ampersand (&). They are used to pass additional information to the server without creating a new route. In Express, query parameters sent by the client are automatically parsed and made available through the `req.query` object. Unlike route parameters, query parameters are optional and not part of the route definition.

Using query parameters:

```

// URL example: http://localhost:3000/search?subject=nodejs&semester=2
app.get('/search', function(req, res) {
  const subject = req.query.subject;    // 'nodejs'
  const semester = req.query.semester;  // '2'
  res.send('Searching for: ' + subject + ' in Semester: ' + semester);
});

```

5.5.3 Common Response Methods in Express

Method	Purpose	Example
<code>res.send(data)</code>	Send a response. Can be a string, object, or Buffer. Automatically sets Content-Type.	<code>res.send('Hello!')</code>
<code>res.json(object)</code>	Send a JSON response. Automatically sets Content-Type to <code>application/json</code> .	<code>res.json({ name: 'Priya' })</code>
<code>res.render(view, data)</code>	Render an HTML template using a template engine (like EJS) and send it.	<code>res.render('index', { title: 'Home' })</code>
<code>res.sendFile(path)</code>	Send a file as the response (for serving HTML files, images, etc.).	<code>res.sendFile('/public/index.html')</code>
<code>res.redirect(url)</code>	Redirect the client to a different URL.	<code>res.redirect('/home')</code>

<code>res.status(code)</code>	Set the HTTP status code of the response. Usually chained with <code>.send()</code> or <code>.json()</code> .	<code>res.status(404).send('Not Found')</code>
-------------------------------	---	--

5.6 Middleware in Express

Definition: Middleware

In Express.js, middleware is a function that has access to the request object (`req`), the response object (`res`), and a special `next()` function in the application's request-response cycle. Middleware functions can execute any code, make changes to the request and response objects, end the request-response cycle, or call the next middleware function in the stack using `next()`. Middleware is used for tasks such as logging, authentication, parsing request bodies, handling errors, and serving static files.

Think of middleware as a series of checkpoints that every request passes through before reaching the route handler. At each checkpoint (middleware), something happens to the request or response. If a checkpoint approves the request, it calls `next()` to pass it on. If not, it can stop the process and send a response immediately.

Example of middleware — a simple request logger:

```
const express = require('express');
const app = express();

// This is a custom middleware function
// It logs details of every incoming request to the console
function requestLogger(req, res, next) {
  const now = new Date().toLocaleTimeString();
  console.log('[ ' + now + ' ] ' + req.method + ' ' + req.url);
  next(); // IMPORTANT: Must call next() to pass control to the next
  middleware/route
}

// app.use() registers middleware to be used for all routes
app.use(requestLogger);

// Built-in middleware: parses incoming JSON request bodies
app.use(express.json());

// Built-in middleware: parses URL-encoded form data
app.use(express.urlencoded({ extended: true }));

app.get('/', (req, res) => res.send('Home Page'));

app.listen(3000);
```

6. Server-Side Rendering with Templating Engines

6.1 What is Server-Side Rendering?

Definition: Server-Side Rendering (SSR)

Server-Side Rendering is a technique in which the HTML content of a web page is generated dynamically on the server before being sent to the client's browser. The server combines an HTML template (which defines the layout and structure) with dynamic data (which changes based on the user or context) and produces a complete, ready-to-display HTML page. This complete HTML is then transmitted to the browser as the HTTP response. The browser simply displays the received HTML without needing to perform any additional processing to render the main content.

To understand why server-side rendering is needed, consider a real-world example. Suppose you are building a student result portal. Each student has a different name, roll number, and marks. You do not want to create a separate HTML file for each student — that would mean creating thousands of files. Instead, you create one HTML template with placeholders, and the server fills in the placeholders with the actual data for each student before sending the page. This is exactly what server-side rendering does.

6.2 What is a Template Engine?

Definition: Template Engine

A template engine is a software component that allows developers to embed dynamic content — such as variables, conditions, and loops — within static HTML templates using special syntax tags. The template engine processes these templates on the server, replaces all the special tags with actual data values, and generates a final HTML document. Popular template engines for Node.js/Express include EJS (Embedded JavaScript), Pug (formerly Jade), Handlebars (hbs), and Mustache.

6.3 Introduction to EJS

Definition: EJS (Embedded JavaScript Templates)

EJS, or Embedded JavaScript, is a simple, lightweight templating language for Node.js that lets you generate HTML with plain JavaScript embedded directly within HTML files. EJS templates are HTML files with the .ejs extension. They look and behave almost exactly like regular HTML, but they support special EJS tags (like `<%= %>` and `<% %>`) that allow you to embed JavaScript expressions and logic directly inside the HTML. EJS is the most straightforward template engine to learn for beginners because it uses familiar JavaScript syntax.

Install EJS using NPM:

Installing EJS:

```
npm install ejs
```

6.4 Setting Up EJS with Express

app.js — Configuring Express to use EJS:

```
const express = require('express');
const app = express();

// Tell Express which template engine to use
// After this line, res.render() will look for .ejs files
app.set('view engine', 'ejs');

// Tell Express where the template files are stored
// By default, Express looks in a folder called 'views'
app.set('views', './views');

// Route: When someone visits /profile, render the 'profile' template
// The second argument to res.render() is an object containing the data
// to pass to the template. The template can access this data using the
// variable names as keys of this object.
app.get('/profile', function(req, res) {
  res.render('profile', {
    studentName: 'Priya Sharma',
    enrollmentNo: '226170307001',
    semester: 2,
    cgpa: 8.75,
    subjects: ['Full Stack Development', 'DBMS', 'Operating Systems',
'DAA']
  });
});

app.listen(3000, () => console.log('Server running!'));
```

6.5 Creating an EJS Template

All EJS template files must be placed inside the views/ folder and must have the .ejs file extension:

File: views/profile.ejs — EJS template with dynamic data

```
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <title>Student Profile</title>
  <style>
```

```

        body { font-family: Arial, sans-serif; margin: 40px; }
        .profile-card { border: 1px solid #ccc; padding: 20px; max-width:
500px; }
        h1 { color: #1F4E79; }
    </style>
</head>
<body>
    <div class='profile-card'>
        <!-- <%= %> outputs the value of a JavaScript variable into the HTML -
->

        <h1>Student Profile: <%= studentName %></h1>
        <p><strong>Enrollment No:</strong> <%= enrollmentNo %></p>
        <p><strong>Semester:</strong> <%= semester %></p>
        <p><strong>CGPA:</strong> <%= cgpa %></p>

        <h2>Subjects Enrolled:</h2>
        <ul>
            <!-- <% %> runs JavaScript code. Here we use a forEach loop -->
            <% subjects.forEach(function(subject) { %>
                <!-- This line runs for each subject in the array -->
                <li><%= subject %></li>
            <% }>; %>
        </ul>
    </div>
</body>
</html>

```

6.6 EJS Tags — Complete Reference

EJS Tag	Name	Purpose	Example
<%= expression %>	Output Tag (Escaped)	Evaluates the JavaScript expression and inserts its value into the HTML. Special HTML characters are automatically escaped for security (< becomes <, etc.).	<%= studentName %> outputs the value of studentName
<%- expression %>	Output Tag (Unescaped)	Evaluates the expression and inserts its value as raw HTML without escaping. Use with caution — only for trusted HTML content.	<%- htmlContent %> renders the HTML directly
<% code %>	Scriptlet Tag	Executes JavaScript code but does not output anything to the HTML. Used for if/else conditions, loops, variable declarations, etc.	<% if (age >= 18) { %> ... <% } %>
<%# comment %>	Comment Tag	An EJS comment that is removed completely during rendering. It will not appear in the final HTML source code.	<%# This is a server-side comment %>

<code><%- include('file') %></code>	Include Tag	Includes the content of another EJS template file. Used for reusable components like headers, footers, and navigation bars.	<code><%- include('header') %></code>
---	-------------	---	---

6.7 Using Conditions in EJS

Using if/else conditions in an EJS template:

```
<!-- views/result.ejs -->
<h1>Result for: <%= studentName %></h1>
<p>Marks Obtained: <%= marks %> / 100</p>

<!-- Use <% %> for JavaScript logic (no output) -->
<% if (marks >= 70) { %>
  <p style='color: green; font-weight: bold;'>Result: PASS (Distinction)</p>
<% } else if (marks >= 35) { %>
  <p style='color: blue;'>Result: PASS</p>
<% } else { %>
  <p style='color: red;'>Result: FAIL</p>
<% } %>
```

7. Static Files in Express

7.1 What are Static Files?

Definition: Static Files

Static files are files that are served directly to the client without any server-side processing or modification. They are called 'static' because their content remains the same regardless of who requests them or when they are requested. In web development, static files typically include CSS stylesheets (which define the visual appearance of web pages), client-side JavaScript files (which add interactivity in the browser), image files (PNG, JPEG, GIF, SVG), font files, PDF documents, and video or audio files. Unlike dynamically generated HTML (which is created differently for each request), static files are pre-built and stored on disk as-is.

7.2 Serving Static Files with `express.static()`

Definition: `express.static()` Middleware

`express.static()` is a built-in middleware function provided by Express for serving static files. When you call `app.use(express.static('folder-name'))`, Express is instructed to make all files inside the specified folder accessible directly via HTTP requests, without needing to define individual routes for each file. Express maps the file paths relative to the static folder directly to URL paths.

Enabling static file serving in Express:

```
const express = require('express');
const app = express();

// This single line makes ALL files in the 'public' folder
// directly accessible via HTTP requests
app.use(express.static('public'));

// After this setup:
// public/css/styles.css   is accessible at:
// http://localhost:3000/css/styles.css
// public/images/logo.png is accessible at:
// http://localhost:3000/images/logo.png
// public/js/app.js       is accessible at: http://localhost:3000/js/app.js

app.get('/', (req, res) => res.render('index'));
app.listen(3000);
```

7.3 Standard Project Folder Structure

When building a Node.js/Express application, it is important to organize your files in a clean, logical way. Here is the recommended folder structure that is widely used by developers:

Standard Express project structure:

```

my-express-app/
├── public/                                ← Static files go here
│   ├── css/
│   │   └── styles.css                    ← Stylesheet for the website
│   ├── js/
│   │   └── script.js                     ← Client-side JavaScript
│   └── images/
│       ├── logo.png
│       └── banner.jpg
├── views/                                ← EJS templates go here
│   ├── index.ejs                         ← Home page template
│   ├── about.ejs
│   └── partials/
│       ├── header.ejs                   ← Reusable header component
│       └── footer.ejs                   ← Reusable footer component
├── routes/                                ← Route files (for large apps)
│   └── studentRoutes.js
├── app.js                                ← Main application file
├── package.json                           ← Project configuration
├── package-lock.json                       ← Exact version lock file
└── .gitignore                             ← Files to exclude from Git

```

Linking static files inside an EJS template:

```

<!-- views/index.ejs -->
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
  <!-- Reference the CSS file using the path RELATIVE to the public folder -
->
  <link rel='stylesheet' href='/css/styles.css'>
</head>
<body>
  <!-- Reference images from the public folder -->
  <img src='/images/logo.png' alt='Company Logo'>
  <h1>Welcome to My Website!</h1>

  <!-- Reference client-side JavaScript from the public folder -->
  <!-- Scripts are placed at the bottom of the body for better performance -
->
  <script src='/js/script.js'></script>
</body>
</html>

```

8. async/await in Node.js

8.1 Understanding Synchronous vs Asynchronous Programming

Definition: Synchronous Programming

In synchronous programming, statements are executed one at a time, in strict sequential order. Each statement must complete before the next one starts. The program waits (blocks) at each step until that step is finished. This is also called blocking execution. While simple to understand, synchronous programming can be very inefficient when dealing with slow operations such as reading large files, making network requests, or querying databases, because the program sits idle waiting for these operations to complete.

Definition: Asynchronous Programming

In asynchronous programming, a long-running operation is initiated, and instead of waiting for it to complete, the program continues executing subsequent code. When the long-running operation eventually finishes, a callback function is invoked, or a Promise is resolved, to handle the result. Asynchronous programming prevents the program from blocking and allows it to remain responsive and handle other tasks while waiting for slow operations. Node.js is built around asynchronous programming because it allows a single server to handle thousands of simultaneous requests efficiently.

Real-World Analogy:

Imagine you are cooking. Synchronous approach: You put rice on the stove, then stand there staring at it until it is cooked (10 minutes), then start chopping vegetables, then cook the curry. You can only do one thing at a time — very slow! Asynchronous approach: You put rice on the stove, and WHILE the rice is cooking, you chop vegetables and prepare the curry sauce. When the rice is done, you handle it. Multiple things progress at the same time — much more efficient! Node.js works like the second approach.

8.2 The Evolution: Callbacks → Promises → async/await

Node.js has gone through three generations of handling asynchronous code, each improving on the previous:

Generation 1: Callbacks (The Old Way)

A callback is a function passed as an argument to another function, to be called when the operation completes. While functional, deeply nested callbacks create what is known as 'Callback Hell' — code that is very difficult to read and maintain:

Callback Hell example (avoid this style):

```
// This is hard to read and maintain - called 'Callback Hell' or 'Pyramid of Doom'
getUserData(userId, function(err, user) {
  if (err) { handleError(err); }
  else {
    getOrders(user.id, function(err, orders) {
      if (err) { handleError(err); }
      else {
        getProducts(orders[0].id, function(err, products) {
          if (err) { handleError(err); }
          else {
            console.log(products); // Deeply nested!
          }
        });
      }
    });
  }
});
```

Generation 2: Promises

Definition: Promise

A Promise is a JavaScript object that represents the eventual result of an asynchronous operation. A Promise can be in one of three states: Pending (the operation is still in progress), Fulfilled (the operation completed successfully and a result is available), or Rejected (the operation failed and an error is available). Promises are created using the Promise constructor and consumed using the .then() method for success and the .catch() method for errors. Promises allow asynchronous code to be chained in a flat, readable manner instead of being deeply nested.

Using Promises with .then() and .catch():

```
getUserData(userId)
  .then(function(user) {
    return getOrders(user.id);
  })
  .then(function(orders) {
    return getProducts(orders[0].id);
  })
```

```
    })
    .then(function(products) {
      console.log(products);
    })
    .catch(function(error) {
      console.log('An error occurred:', error);
    });
// Better than callbacks, but still verbose
```

Generation 3: async/await (The Modern Way)

Definition: async/await

async/await is syntactic sugar built on top of Promises that allows asynchronous, Promise-based code to be written in a style that looks and reads like synchronous code. The 'async' keyword is placed before a function declaration to indicate that the function always returns a Promise. The 'await' keyword can only be used inside an async function; it pauses the execution of the function until the awaited Promise is resolved (or rejected), and then returns the resolved value. The try/catch block is used to handle errors with async/await, just like in synchronous code. async/await was introduced in ECMAScript 2017 (ES8).

The same operation using async/await — clean and readable:

```
// The 'async' keyword makes this function asynchronous
async function loadData() {
  try {
    // 'await' pauses execution here until getUserData() resolves
    // The result is stored in 'user', just like synchronous code
    const user = await getUserData(userId);

    // 'await' pauses again until getOrders() resolves
    const orders = await getOrders(user.id);

    const products = await getProducts(orders[0].id);

    console.log(products); // Clean, flat, easy to read!

  } catch (error) {
    // All errors from any of the above await calls
    // are caught here in one place
    console.log('An error occurred:', error);
  }
}

loadData(); // Call the async function
```

8.3 Rules and Key Points of async/await

- **Rule 1:** The 'async' keyword must be placed before the function keyword when declaring an asynchronous function: `async function myFunction() { ... }` or `const myFunction = async () => { ... }`
- **Rule 2:** The 'await' keyword can ONLY be used inside a function that has been declared with the 'async' keyword. Using await outside an async function will cause a `SyntaxError`.
- **Rule 3:** An async function ALWAYS returns a Promise, even if you return a plain value inside it. The plain value is automatically wrapped in a resolved Promise.
- **Rule 4:** Always wrap your await statements inside a `try { } catch (error) { }` block to handle potential errors gracefully.
- **Rule 5:** await only works with Promises (or functions that return Promises). Applying await to a non-Promise value simply returns the value immediately.

8.4 async/await with Express Routes

Using async/await in an Express route handler:

```
const express = require('express');
const fs = require('fs').promises; // Use the promise-based version of fs
const app = express();

// Making a route handler async allows us to use await inside it
app.get('/read-file', async function(req, res) {
  try {
    // Instead of using a callback, we await the file read operation
    const content = await fs.readFile('./data.txt', 'utf8');
    res.send(content);
  } catch (error) {
    // If the file doesn't exist or can't be read, send a 500 error
    console.error('Error reading file:', error.message);
    res.status(500).send('Error: Could not read the requested file.');
```

9. Fetching JSON Data from Express

9.1 What is JSON?

Definition: JSON (JavaScript Object Notation)

JSON stands for JavaScript Object Notation. It is a lightweight, human-readable, and language-independent data interchange format used to store and transmit structured data between a server and a client. JSON is derived from JavaScript object literal syntax, but it is supported by virtually all modern programming languages. In JSON, data is represented as key-value pairs enclosed in curly braces {}. Keys must be strings enclosed in double quotes. Values can be strings, numbers, booleans, arrays, null, or nested JSON objects. JSON has become the standard format for data exchange in modern web APIs.

JSON Example — A student record:

```
{
  "name": "Priya Sharma",
  "enrollmentNo": "226170307001",
  "semester": 2,
  "cgpa": 8.75,
  "isActive": true,
  "address": {
    "city": "Surat",
    "state": "Gujarat"
  },
  "subjects": ["Full Stack Development", "DBMS", "Operating Systems"]
}
```

JSON Data Type	Description	Example
String	Text value — must be in double quotes	"name": "Priya"
Number	Integer or decimal — no quotes	"age": 22
Boolean	true or false — no quotes	"isActive": true
null	Represents an empty or unknown value	"middleName": null
Array	An ordered list of values — in square brackets []	"subjects": ["DBMS", "OS"]
Object	A nested JSON object — in curly braces {}	"address": { "city": "Surat" }

9.2 What is an API?

Definition: API (Application Programming Interface)

An API, or Application Programming Interface, is a set of rules, protocols, and definitions that allows one software application to communicate with another. In web development, an API typically refers to a Web API — a server that exposes specific URL endpoints (called API endpoints) that clients can send HTTP requests to in order to retrieve data, create records, update information, or delete records. Web APIs commonly exchange data in JSON format. A RESTful API is the most common type, which follows a set of principles called REST (Representational State Transfer).

When you use a mobile app that shows the weather, the app is calling a weather API to get the current data. When you log in to a website using your Google account, the website is calling Google's API. APIs allow different systems to talk to each other in a standardized way.

9.3 Sending JSON Responses with Express

In Express, sending a JSON response is extremely simple. The `res.json()` method automatically converts a JavaScript object or array to a JSON string and sets the Content-Type header to `application/json`:

app.js — Sending JSON from Express routes:

```
const express = require('express');
const app = express();

// Sample data - in a real application, this comes from a database
const students = [
  { id: 1, name: 'Priya Sharma', semester: 2, cgpa: 8.75 },
  { id: 2, name: 'Ravi Patel', semester: 2, cgpa: 7.90 },
  { id: 3, name: 'Anita Desai', semester: 2, cgpa: 9.10 },
  { id: 4, name: 'Arjun Mehta', semester: 2, cgpa: 8.20 },
];

// API Endpoint 1: GET all students
// URL: http://localhost:3000/api/students
app.get('/api/students', function(req, res) {
  res.json(students); // Sends the entire array as JSON
});

// API Endpoint 2: GET a single student by ID
// URL: http://localhost:3000/api/students/2
app.get('/api/students/:id', function(req, res) {
  const requestedId = parseInt(req.params.id); // Convert string to integer
  const student = students.find(s => s.id === requestedId);

  if (student) {
    // Student found - send it with 200 OK
    res.status(200).json(student);
  } else {
```

```
        // Student not found – send 404 with an error message
        res.status(404).json({ error: 'Student with ID ' + requestedId + ' not
found.' });
    }
});

app.listen(3000, () => console.log('API Server running at
http://localhost:3000'));
```

9.4 Receiving JSON Data from the Client (POST Request)

When a client (browser, mobile app, or another server) sends data to the server via a POST, PUT, or PATCH request, the data is included in the request body. To access this data in Express, you must first enable the `express.json()` middleware, which parses the incoming JSON body and makes it available through `req.body`:

Receiving and processing JSON data sent by a client:

```
const express = require('express');
const app = express();

// CRITICAL: This middleware MUST be added before any routes that use req.body
// It tells Express to parse incoming JSON request bodies
app.use(express.json());

let students = [
  { id: 1, name: 'Priya Sharma', cgpa: 8.75 }
];

// POST Endpoint: Add a new student
// The client sends JSON in the request body: { 'name': 'Ravi', 'cgpa': 7.9 }
app.post('/api/students', function(req, res) {
  const newStudent = req.body; // Access the parsed JSON body

  // Validate: check that required fields are provided
  if (!newStudent.name || !newStudent.cgpa) {
    return res.status(400).json({ error: 'Name and CGPA are required.' });
  }

  // Assign a new ID and add to the array
  newStudent.id = students.length + 1;
  students.push(newStudent);

  // 201 Created status – the resource was successfully created
  res.status(201).json({
    message: 'Student added successfully!',
    student: newStudent
  });
});

app.listen(3000, () => console.log('Server running!'));
```

9.5 Fetching JSON from the Browser (Client-Side)

Definition: Fetch API

The Fetch API is a modern, Promise-based browser interface that provides a way to make HTTP requests from client-side JavaScript code (running in the browser). It replaces the older XMLHttpRequest (XHR) method and provides a cleaner, more flexible API for making GET, POST, PUT, DELETE, and other HTTP requests. The Fetch API is built into all modern browsers and does not require any additional library. It returns a Promise that resolves to a Response object, from which the response body can be extracted in various formats including JSON, text, and binary data.

Fetching JSON from an Express API using the browser Fetch API:

```
// This code runs in the BROWSER (inside an HTML file's <script> tag)

// Async function to load and display all students from our API
async function loadAllStudents() {
  try {
    // fetch() sends a GET request to the URL and returns a Promise
    const response = await fetch('http://localhost:3000/api/students');

    // Check if the request was successful
    if (!response.ok) {
      throw new Error('Network response was not ok. Status: ' +
        response.status);
    }

    // response.json() parses the JSON body and returns a Promise
    const students = await response.json();

    // Now 'students' is a JavaScript array that we can use
    console.log('Total students:', students.length);

    // Display the data in the HTML page
    const container = document.getElementById('studentList');
    students.forEach(function(student) {
      const li = document.createElement('li');
      li.textContent = student.name + ' - CGPA: ' + student.cgpa;
      container.appendChild(li);
    });

  } catch (error) {
    console.error('Failed to fetch students:', error.message);
  }
}

// Call the function when the page loads
window.onload = loadAllStudents;
```